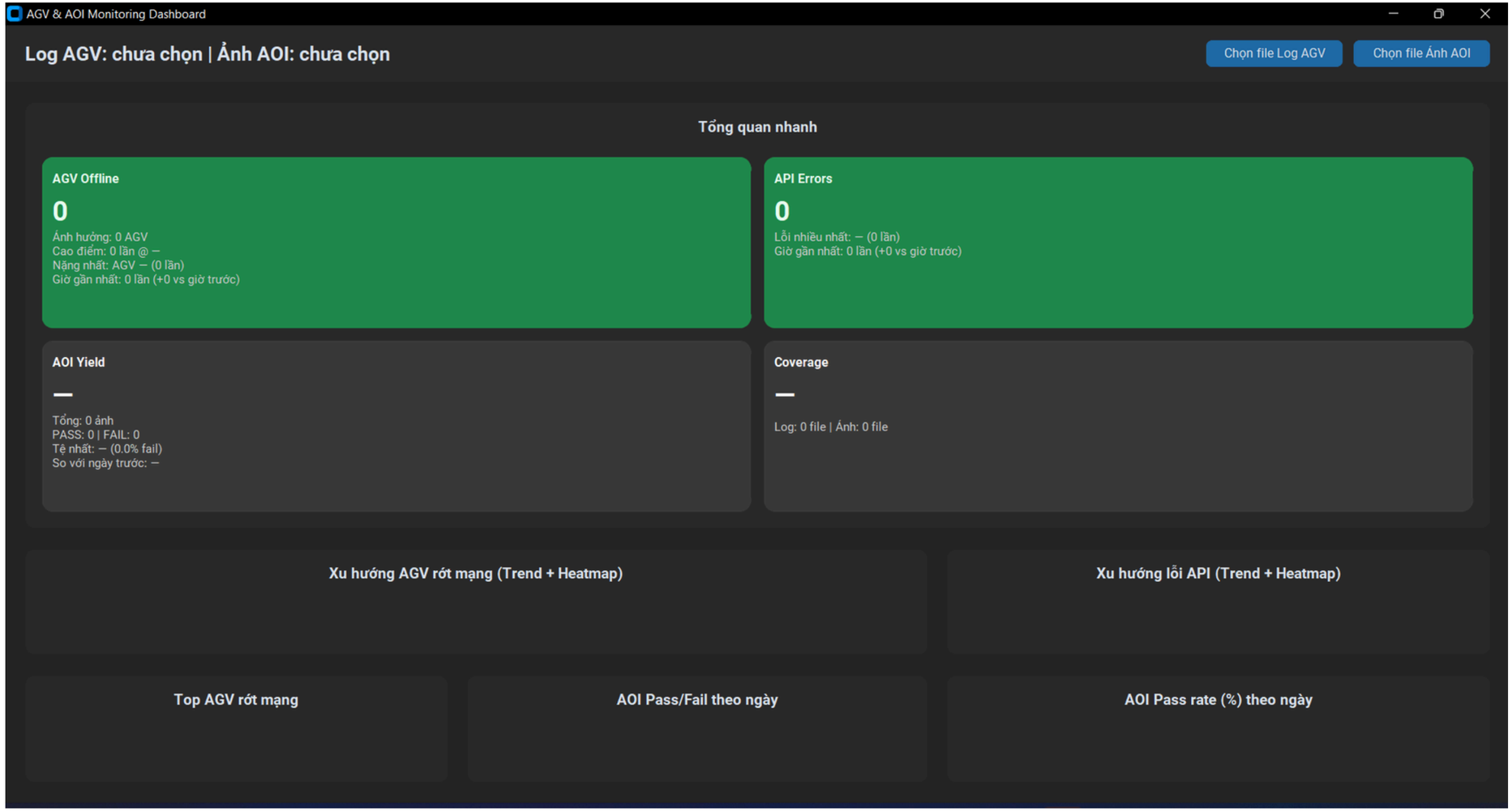
BG6: AOI & AGV DASHBOARD



Presented By:
Pham Minh Quang
范明光

Pegatron Corp

System Goals

- Quick Alerts: Spot AGV network drops and API errors fast.
- Quality Check: Track AOI quality via PASS/FAIL rates.
- Smart KPIs: Clear dashboards for fast decision-making.
- Multi-select: Support picking MANY log files and MANY images at once.

Key Benefits

- Quick Look: See "Status OK" or "System Error" instantly.
- Find Hotspots: Identify peak error times by hour or day.
- Top Issues: Know exactly which AGV or API is causing the most trouble.

3-File Architecture

- main.py: UI/Dashboard, KPIs, charts, and hover tooltips.
- agv.py: AGV log parser; summarizes offline status, API errors, and coverage.
- aoi.py: AOI image parser; summarizes daily PASS/FAIL rates.

Why Separate Modules?

- Easy Maintenance: Fix bugs in one part without breaking others.
- Scalability: Add new features or hardware types easily.
- Independent Testing: Test each module (AGV, AOI, UI) on its own.

```
factory_dashboard/
 ├── main.py #  Main UI & Dashboards
 ├── agv.py # AGV Log Parsing & Metrics
 ├── aoi.py # AOI Image Parsing & Metrics
```

Data Flow

1. Select Logs: User picks AGV log files (multi-select).
2. Select Images: User picks AOI image files (multi-select).
3. Data Parsing: AGV/AOI Parsers process the raw data.
4. UI Update: main.py updates KPIs, Chart Grid, and Tooltips.
5. Dashboard View: Shows system overview and performance trends.

main.py

```python
def load_agv_logs(self):
    file_paths = filedialog.askopenfilenames( ... )
    df_offline, df_api, system_events, coverage = agv.parse_agv_logs(file_paths)
    self.df_offline = df_offline
    self.df_api = df_api
    self.coverage = coverage
    self.draw_agv_chart(self.df_offline)
    self.draw_api_chart(self.df_api)
    self.draw_offline_trend(self.df_offline)
    self.update_kpi()

def load_aoi_images(self):
    file_paths = filedialog.askopenfilenames( ... )
    self.df_aoi = aoi.parse_aoi_images(file_paths)
    self.draw_aoi_chart(self.df_aoi)
    self.draw_aoi_rate_chart(self.df_aoi)
    self.update_kpi()
```

agv.py Functions

1) _normalize_log_inputs(...)

- Input Normalization: Convert inputs (single file, multiple files, or a folder) into a valid list of .txt files.

2) parse_agv_logs(...)

- Read Logs: Scan each log line.
- Capture Timestamps: Grab the exact time of each event.
- Extraction: Pull data for:
  - AGV Offline: Connection drops.
  - API Errors: Failed API calls.
  - System Events: Related system activities.
- Data Aggregation (Summary):
  - df_offline: Count by Hour and AGV.
  - df_api: Count by Hour and API.
  - coverage: Time range (min/max) and file list.

agv.py

```python
def parse_agv_logs(paths_or_folder):
    time_pattern = r'(\\d{4}-\\d{2}-\\d{2}\\s+\\d{2}:\\d{2}:\\d{2})(?:\\.\\d+)?'
    ...
    if offline_match:
        offline_events.append({'TS': timestamp, 'Hour': ..., 'AGV': ...})
    if "无法连接到远程服务器" in line or "Unable to connect to the remote server" in line:
        api_errors.append({'TS': timestamp, 'Hour': ..., 'API': ...})
    ...
    coverage = {"ts_min": ts_min_all, "ts_max": ts_max_all, "files": len(file_paths)}
    return df_offline, df_api, system_events, coverage
```

aoi.py Functions

1) _normalize_image_inputs(...)

- Image Normalization: Standardize image inputs (supporting .png, .jpg, .jpeg, .bmp).

2) parse_aoi_images(...)

- Read Filenames: Scan names of image files.
- Logic Rules:
  - PASS: If filename contains "ALL PASS".
  - FAIL: If filename contains "FAIL".
- Date Extraction: Extract date information directly from the filename.
- Data Aggregation: Summarize results into a daily PASS/FAIL table.

aoi.py

```python
def parse_aoi_images(paths_or_folder):
    for file_path in file_paths:
        filename = os.path.basename(file_path)
        status = 'PASS' if 'ALL PASS' in filename.upper() else 'FAIL' if 'FAIL' in filename.upper() else 'UNKNOWN'
        date_match = re.search(r'\\d{8}', filename)
        ...
    summary = df.groupby(['Date', 'Status']).size().unstack(fill_value=0).reset_index()
    return summary
```

Dashboard Grid Layout

Row 0 :

- Quick Overview: KPI Cards (Total metrics at a glance).

Row 1:

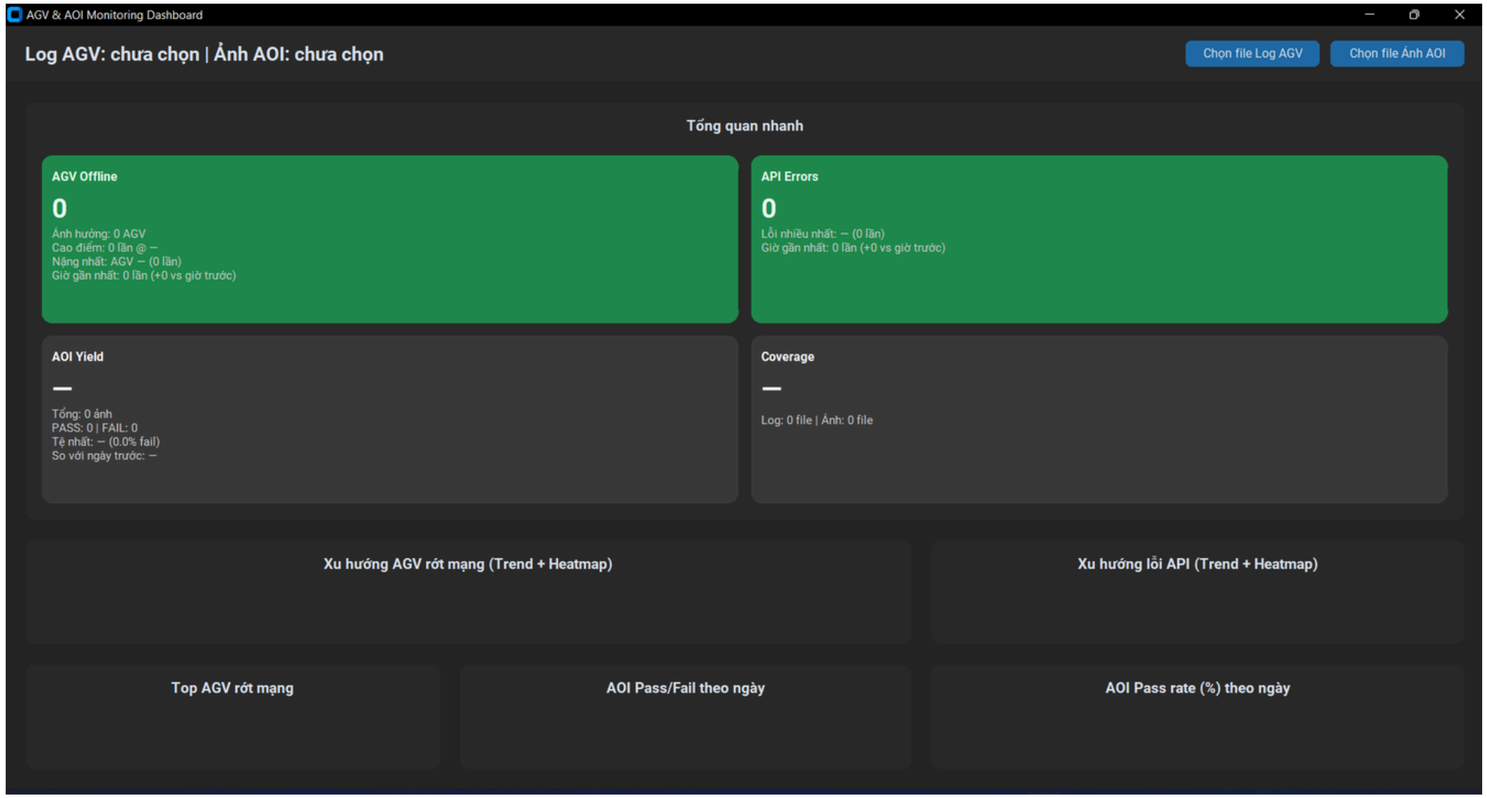
- Left (2 columns): AGV Network Drop Trends (Line Chart + Heatmap).
- Right (1 column): API Error Trends (Line Chart + Heatmap).

Row 2:

- Top AGV Drops: Identify the most unstable AGVs.
- Daily AOI PASS/FAIL: Stacked bar chart for volume.
- Daily AOI Pass Rate (%): Line chart for quality tracking.

Visual Meaning

- Trend Charts: Track increases or decreases over time.
- Heatmaps: Spot "peak hours" or "hot zones" for errors.
- Top AGV: Pinpoint the primary units causing the most impact.
- AOI Charts: Visualize the balance between PASS/FAIL and overall quality.



1) AGV Offline

- Total Offlines
- Impacted AGVs
- Peak Hour
- Worst AGV
- Current vs. Previous Hour

2) API Errors

- Total Errors
- Top Failed Endpoint
- Current vs. Previous Hour

3) AOI Yield

- Pass Rate (%)
- Total PASS / FAIL
- Worst Day
- Change vs. Yesterday

4) Coverage

- Time Range (From - To)
- Files Analyzed

main.py

```python
def update_kpi(self):
    offline_total = int(self.df_offline['Count'].sum()) if ... else 0
    api_total = int(self.df_api['Count'].sum()) if ... else 0
    aoi_pass = int(self.df_aoi['PASS'].sum()) if ... else 0
    aoi_fail = int(self.df_aoi['FAIL'].sum()) if ... else 0
    ...
    self._kpi_set("agv", ... )
    self._kpi_set("api", ... )
    self._kpi_set("aoi", ... )
    self._kpi_set("cover", ... )
```

Hover Implementation

- Bar Charts: Top AGVs, AOI PASS/FAIL.
- Line Charts: Trends, Pass Rates.
- Heatmaps: Offline & API errors.

On-Hover Information

- Chart Context: Chart name and meaning.
- Data Points: Exact values at the pointer.
- Time Context: Specific date or hour.

Benefits

- No Guessing: Users don't have to "guess" the data.
- Quick Demo: Clear and fast visual explanation.

main.py

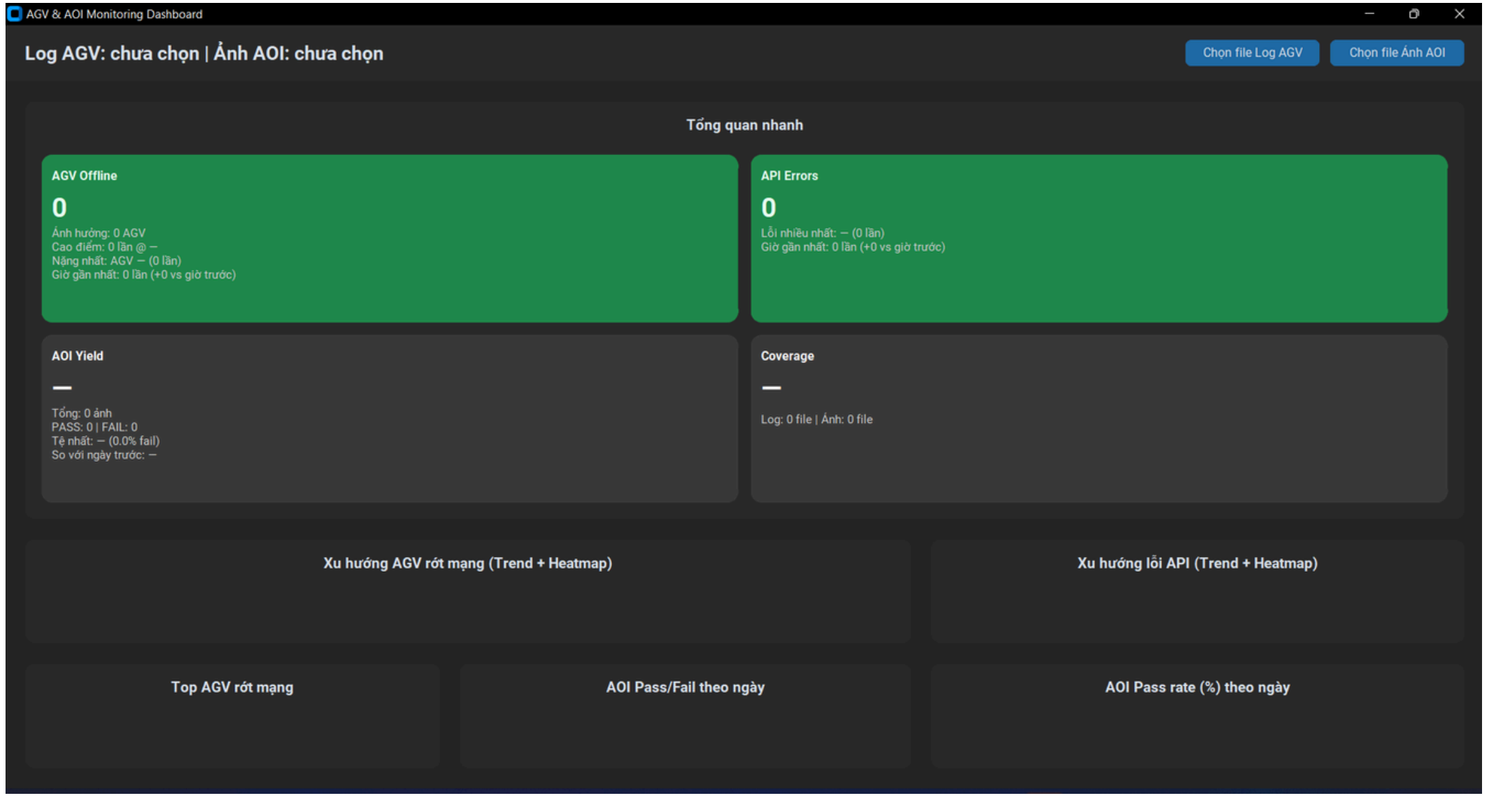
```python
annot = ax.annotate( ... , annotation_clip=False)
def on_move(event):
    ...
    annot.set_text(
        "Heatmap loi API\\n"
        "Y nghia: tong so loi API trong 1 gio.\\n\\n"
        f"Ngay: {day}\\nGio: {hour}:00\\nAPI errors: {value}"
    )
    annot.set_visible(True)
```

Grid Design Goals

- **Top Row:** Highlighted KPIs for instant status checks.
- **Middle Row:** Trends and Heatmaps to track issues over time.
- **Bottom Row:** Summary charts for final conclusions.

main.py

```python
self.frame_kpi = self.create_chart_frame(0, 0, "Tong quan nhanh", colspan=3)
self.frame_offline_trend = self.create_chart_frame(1, 0, "Xu huong AGV ...", colspan=2)
self.frame_api_trend = self.create_chart_frame(1, 2, "Xu huong loi API ...")
self.frame_agv = self.create_chart_frame(2, 0, "Top AGV rot mang")
self.frame_aoi = self.create_chart_frame(2, 1, "AOI Pass/Fail theo ngay")
self.frame_aoi_rate = self.create_chart_frame(2, 2, "AOI Pass rate (%) theo ngay")
```



Current Limitations & Operational Status

- **Access Restrictions:** The internal database is under strict corporate control. We do not have the necessary permissions to establish a direct connection.
- **Technical Scope:** Implementing an automated sync requires complex protocols that are beyond our current technical capacity.
- **Current Workflow:** Due to these constraints, we are currently limited to manual data collection (collecting files and importing them into the system).
- **Operational Reality:** This manual process is the only feasible way to move forward under our existing resources and access levels.
- Resource Constraints: Direct integration would require specialized IT support and infrastructure changes that are not available at this stage.

THE END

Presented By:
Pham Minh Quang
范明光

Pegatron Corp